UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS
LENGUAJES FORMALES Y DE PROGRAMACION
INGENIERO VIVIAN CAMPOS GONZALES
TUTOR: LUISA ORTIZ ROMERO

LFP_A-



PROYECTO #1

Manual Técnico

CARNE	NOMBRE
202300697	Wilson Wilfredo Pérez Otzoy

- **Requerimientos mínimos del entorno de Desarrollo**
    - IntelliJ / Neatbeans
        - 2GB de Ram
        - Procesador de 1.6Hz o mayor.
    - Java
        - Maven
        - Librerias
            - Swing
            - Java.io
            - ArraysList
            - List
            - BufferedWriter
            - File
            - FileWriter
        - Git
        - GitHub
    - Windows 10
- **Clases**
    - Token
    - Error
    - Métodos de setters y getters
    - Método toString()

FUNCIONES Y/O MÉTODOS FUNCIONALES DEL PROYECTO

- **agregarToken(carácter, token):** Este método permite agregar los tokens reconocidos en el lenguaje a una lista dinámica.

```
//agrego token a mi lista de tokens
public void agregarToken(String caracter, String token, int
 linea, int columna){  13 usages  WilOtzoy
    this.tokens.add(new Token(caracter, linea, columna, token));
    this.buffer = "";
}
```

- **agregarToken(carácter, token):** Este método permite agregar los tokens reconocidos en el lenguaje a una lista dinamica.

```
//muestro como error los caracteres no aceptados en mi lenguaje
public void agregarError(String caracter, int linea, int
 columna){  2 usages  WilOtzoy
    this.errores.add(new Error(caracter,  descripcion: "Caracter
     "+caracter+" no reconocido en el lenguaje", linea,
     columna));
    this.buffer = "";
}
```

- **analizar(cadena):** Este método permite analizar el contenido de la cadena cargada en el archivo, para poder determinar a que estado del autómata pertenece determinar los tokens del lenguaje.

```
public void analizar(StringBuilder cadena)
{  1 usage  WilOtzoy *
    this.tokens.clear();
    this.errores.clear();
    this.estado = 0;
    this.i = 0;
    this.buffer = "";

    while(this.i < cadena.length()){
        //System.out.println("wile cadena");
        if(this.estado == 0){
            S0(cadena.charAt(this.i));
        }
        else if(this.estado == 1){
            S1(cadena.charAt(this.i));
        }

        else if(this.estado == 3){
            S3(cadena.charAt(this.i));
        }
        else if(this.estado == 4){
            S4(cadena.charAt(this.i));
        }
        i = i+1;
    }
}
```

- **S0(caracter)**: Este método permite reconocer los tokens del lenguaje y sus lexemas, reconoce símbolos también que son parte del S2 (el código muestra solo un fragmento de los tokens, pero siguen la misma lógica todos los tokens).



```
// estados del AFD
public void S0(char caracter){  1 usage   WilOtzoy *

    if(caracter == '{'){
        this.agregarToken(String.valueOf(caracter),
          token: "llaveApertura", this.linea, this.columna);
        this.columna+=1;
    }else if(caracter == '}'){
        this.agregarToken(String.valueOf(caracter),
          token: "llaveCierre", this.linea, this.columna);
        this.columna+=1;
    }else if(caracter == ':'){
        this.agregarToken(String.valueOf(caracter),  token: "dosPuntos",
         this.linea, this.columna);
        this.columna+=1;
    }else if(caracter == ','){
        this.agregarToken(String.valueOf(caracter),  token: "coma",
         this.linea, this.columna);
        this.columna+=1;
    }else if(caracter == '('){
        this.agregarToken(String.valueOf(caracter),
          token: "parentesisApertura", this.linea, this.columna);
        this.columna+=1;
```

- **S1(carácter):** Este estado permite reconocer cadenas que los podemos clasificar como palabras reservadas o identificadores.



```
public void S1(char caracter){  1 usage   WilOtzoy *
    if(Character.isLetter(caracter) || Character.isDigit(caracter)){
        this.buffer+= caracter;
        this.columna+=1;
    }else{
        // evaluado las palabrase reservadas en el automata
        if(this.buffer.equals("descripcion") || this.buffer.equals("estados") ||
         this.buffer.equals("alfabeto") || this.buffer.equals("inicial") ||
         this.buffer.equals("finales") || this.buffer.equals("transiciones")){
            this.agregarToken(this.buffer,  token: "palabraReservada",linea, columna);
        }else{
            this.agregarToken(this.buffer,  token: "identificador",linea, columna);
        }

        this.estado = 0;
        this.columna+=1;
        this.i -= 1; //Regreso el índice pues ya terminé de reconocer el
         identificador
    }
}
```

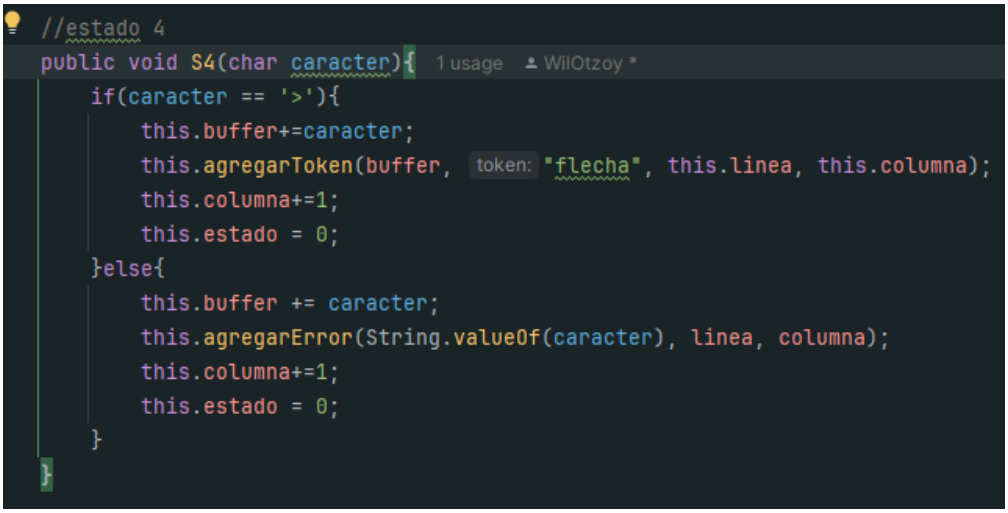
- **S3(caracter):** Este estado nos permite reconocer cadenas en el lenguaje seguidas por comillas.



```
//estado 3
public void S3(char caracter){  1 usage  WilOtzoy *
    if(caracter != '"'){
        this.buffer+=caracter;
        this.columna+=1;
    }else{
        //Aceptación
        this.buffer+=caracter;
        this.agregarToken(this.buffer, token: "cadena",linea, columna);
        this.estado = 0;
        this.columna+=1;

    }
}
```

- **S4(carácter):** Este estado nos permite reconocer las flechas dentro del lenguaje y agregarlos a la lista de tokens, de lo contrario nos redirecciona al estado S0 para seguir reconociendo tokens.



```
//estado 4
public void S4(char caracter){  1 usage  WilOtzoy *
    if(caracter == '>'){
        this.buffer+=caracter;
        this.agregarToken(buffer,  token: "flecha", this.linea, this.columna);
        this.columna+=1;
        this.estado = 0;
    }else{
        this.buffer += caracter;
        this.agregarError(String.valueOf(caracter), linea, columna);
        this.columna+=1;
        this.estado = 0;
    }
}
```

- **Inicializar():** Este método lo mando a llamar en el botón de analizar archivo, su funcionalidad es de darle al usuario la opción de elegir el archivo a conveniencia a través de un explorador y analizar su contenido.

```java
public void inicializar(){ 1 usage  WilOtzoy *
    JFileChooser fileChooser = new JFileChooser();
    int selection = fileChooser.showOpenDialog( parent: null);
    if (selection == JFileChooser.APPROVE_OPTION) {
        File archivoSeleccionado = fileChooser.getSelectedFile();
        String ruta = archivoSeleccionado.getAbsolutePath();
        try {
            File archivo = new File(ruta);
            StringBuilder content = new StringBuilder();
            if (archivo.exists()) {
                BufferedReader br = new BufferedReader(new FileReader(archivo));
                String linea;
                while ((linea = br.readLine()) != null) {
                    content.append(linea).append("\n");
                }
                analizadorLexico.analizar(content);
                textArea.setText(content.toString());

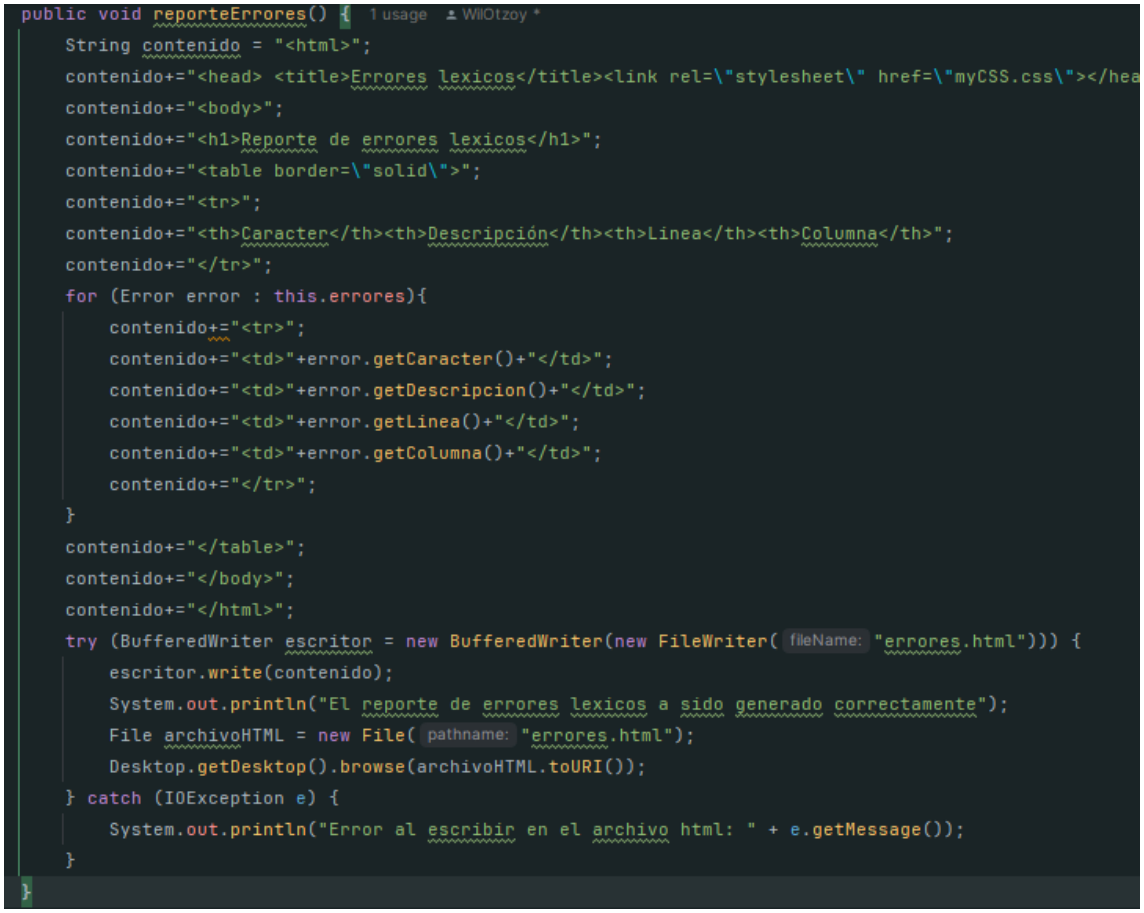
            } else {
                System.out.println("El archivo no existe.");
            }
        }
        catch (IOException e) {
            System.out.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}
```

- **reporteTokens():** Este método permite generar un archivo html con todos los tokens reconocidos en el lenguaje detallando su lexema y en que línea y columna fueron encontrados.

```java
public void reporteTokens() { 1 usage  WilOtzoy *
    String contenido = "<html>";
    contenido+="<head> <title>Reporte tokens</title><link rel=\"stylesheet\" href=\"myCSS.css\"></head>";
    contenido+="<body>";
    contenido+="<h1>Reporte de tokens</h1>";
    contenido+="<table border=\"solid\">";
    contenido+="<tr>";
    contenido+="<th>Token</th><th>Lexema</th><th>Linea</th><th>Columna</th>";
    contenido+="</tr>";
    for (Token token : this.tokens){
        contenido+="<tr>";
        contenido+="<td>"+token.getTipo()+"</td>";
        contenido+="<td>"+token.getLexema()+"</td>";
        contenido+="<td>"+token.getLinea()+"</td>";
        contenido+="<td>"+token.getColumna()+"</td>";
        contenido+="</tr>";
    }
    contenido+="</table>";
    contenido+="</body>";
    contenido+="</html>";
    try (BufferedWriter escritor = new BufferedWriter(new FileWriter( fileName: "tokens.html"))) {
        escritor.write(contenido);
        System.out.println("El reporte de tokens a sido generado correctamente");
        File archivoHTML = new File( pathname: "tokens.html");
        Desktop.getDesktop().browse(archivoHTML.toURI());
    } catch (IOException e) {
        System.out.println("Error al escribir en el archivo html: " + e.getMessage());
    }
}
```

Font size: 11pt Reset to 13pt

- **reporteErrores():** Este método permite generar un archivo html con todos los errores léxicos en el lenguaje detallando su descripción del carácter y en que línea y columna fueron encontrados.



```
public void reporteErrores() {  1 usage   WillOtzoy *
    String contenido = "<html>";
    contenido+="<head> <title>Errores lexicos</title><link rel=\"stylesheet\" href=\"myCSS.css\"></hea
    contenido+="<body>";
    contenido+="<h1>Reporte de errores lexicos</h1>";
    contenido+="<table border=\"solid\">";
    contenido+="<tr>";
    contenido+="<th>Caracter</th><th>Descripción</th><th>Linea</th><th>Columna</th>";
    contenido+="</tr>";
    for (Error error : this.errores){
        contenido+="<tr>";
        contenido+="<td>"+error.getCaracter()+"</td>";
        contenido+="<td>"+error.getDescripcion()+"</td>";
        contenido+="<td>"+error.getLinea()+"</td>";
        contenido+="<td>"+error.getColumna()+"</td>";
        contenido+="</tr>";
    }
    contenido+="</table>";
    contenido+="</body>";
    contenido+="</html>";
    try (BufferedWriter escritor = new BufferedWriter(new FileWriter( fileName: "errores.html"))) {
        escritor.write(contenido);
        System.out.println("El reporte de errores lexicos a sido generado correctamente");
        File archivoHTML = new File( pathname: "errores.html");
        Desktop.getDesktop().browse(archivoHTML.toURI());
    } catch (IOException e) {
        System.out.println("Error al escribir en el archivo html: " + e.getMessage());
    }
}
```